

SPENWYSE

FINANCE ENGINE UNIT TESTING

Interview Preparation & Study Guide

Implementing robust Jest tests for daily spending pools, safe-to-spend limits, and mid-cycle resource allocations.

Key Highlights Covered:

- ' Deterministic Time Mocking using Jest Fake Timers & setSystemTime
- ' Financial Math & Formula Validation (Safe to Spend Today)
- ' Zero/Negative Balance and Over-budget Capping Edge Cases
- ' Dynamic Mid-Cycle Commitment and Goal-Locker Recalculation
- ' February Leap Year and Month-End Boundary Checks

1. Core Math & Calculation Engine

Understanding the business logic behind SpenWyse calculations

SpenWyse guides users towards disciplined spending by calculating a dynamic daily budget known as the "Safe to Spend Today" limit. Rather than enforcing strict categorical caps, the app treats all uncommitted funds as a single flexible pool, distributing it evenly over the remaining days of the current month.

The Core Formulas

1. Flexible Pool Calculation

The Flexible Pool represents the amount of money a user has left that is not yet spent or locked in goals (commitments).

$$\text{Flexible Pool} = \text{Monthly Income} - \text{Commitments} - \text{Spent}$$

2. Remaining Days in Month

Remaining days includes the current day itself. It dynamically updates as the calendar moves.

$$\text{Remaining Days} = (\text{Total Days in Current Month} - \text{Current Day of Month}) + 1$$

3. Safe to Spend Today (Daily Limit)

This is the division of the Flexible Pool by the remaining days of the month. If the Flexible Pool is zero or negative (over-budget), the limit is capped at 0.

$$\text{Safe to Spend Today} = \text{Math.max}(\text{Math.floor}(\text{Flexible Pool} / \text{Remaining Days}), 0)$$

Scenario walkthrough

Suppose a user has a monthly income of ₹30,000. It is July 15th (17 days left in July). They have spent ₹15,000 so far and locked ₹10,000 in goals (commitments).

- Flexible Pool = ₹30,000 - ₹10,000 - ₹15,000 = ₹15,000
- Safe to Spend Today = ₹15,000 / 17 days = ₹882.35 (floored to ₹882)

2. Unit Testing Strategy

How we ensure calculation correctness and deterministic test runs

Unit testing calculations that depend on the current calendar date presents a major challenge: the test results will change depending on the day they are executed. If a test runs on July 15th, the remaining days will be 17. If it runs on July 25th, the remaining days will be 7, causing assertions to fail.

Mocking System Time

To achieve determinism, we use Jest's fake timer utilities. This allows us to "time travel" to specific dates and verify how the system behaves on those exact days, regardless of the actual date on the host machine.

How it works in Jest:

- `jest.useFakeTimers()` tells Jest to intercept native date/time functions.
- `jest.setSystemTime(date)` sets the system clock to a fixed, specified date.
- `jest.useRealTimers()` restores the native clock functionality after each test.

Here is the setup we implement in our test suite:

```
beforeEach(() => {  
  jest.useFakeTimers();  
});  
  
afterEach(() => {  
  jest.useRealTimers();  
});
```

Deterministic assertions

By setting the system time, we lock the value of `remainingDays` to a known constant. For example, setting the clock to July 15th guarantees that remaining days will always evaluate to exactly 17, making our mathematical assertions stable and reliable:

```
test('should return 17 remaining days on July 15th', () => {  
  jest.setSystemTime(new Date('2026-07-15T12:00:00Z'));  
  expect(getRemainingDaysInMonth()).toBe(17);  
});
```

3. Critical Edge Cases Tested

Robustness testing for unusual financial states and calendar configurations

A reliable financial engine must handle extreme input values, zero states, and calendar variances. Our unit test suite targets three primary edge cases:

1. Zero & Negative Balances

If a user overspends ($\text{Spent} > \text{Income}$) or locks more than they have, the Flexible Pool becomes negative. The Safe to Spend Today limit must not show a negative value; it must be gracefully capped at zero.

```
// Spent (20,000) + Commitments (15,000) exceeds Income (30,000)
// Flexible Pool = -5,000. Daily Limit should be 0.
const snapshot = calculateFinanceSnapshot({
  income: 30000,
  commitments: 15000,
  spent: 20000,
});
expect(snapshot.flexiblePool).toBe(-5000);
expect(snapshot.dailySpendLimit).toBe(0);
```

2. Mid-Cycle Allocation Changes

Users frequently lock money into goals (commitments) or make sudden transactions in the middle of a month. The daily limit must recalculate immediately and adjust. Our tests verify that changing commitments on July 15th successfully reduces the daily limit from $11,176$ down to 1588 :

```
// Commitments change mid-cycle from 5,000 to 15,000
let snapshot = calculateFinanceSnapshot({ income: 30000, commitments: 5000, spent: 5000 });
expect(snapshot.dailySpendLimit).toBe(1176); // (20,000 / 17)

snapshot = calculateFinanceSnapshot({ income: 30000, commitments: 15000, spent: 5000 });
expect(snapshot.dailySpendLimit).toBe(588); // (10,000 / 17)
```

3. Calendar Boundaries (Leap Years & Last Days)

We verify leap year February (29 days) vs non-leap year February (28 days), and the last day of a month (where remaining days = 1, meaning the daily limit equals the entire flexible pool).

```
// Feb 15th, 2028 (Leap Year) -> 29 - 15 + 1 = 15 remaining days
jest.setSystemTime(new Date('2028-02-15T12:00:00Z'));
expect(getRemainingDaysInMonth()).toBe(15);

// July 31st (Month-end) -> 1 remaining day
jest.setSystemTime(new Date('2026-07-31T12:00:00Z'));
expect(getRemainingDaysInMonth()).toBe(1);
```

4. Mock Interview Q&A

How to talk about this project work in technical and behavioral interviews

Q1: How did you test calculations that depend on the current date?

A: "I used Jest's fake timer utilities. I implemented `jest.useFakeTimers()` and `jest.setSystemTime()` to lock the system clock to specific dates (like July 15th or February 28th). This made the remaining days in a month a fixed constant, allowing the math to be fully deterministic and reproducible on any system."

Q2: What boundary conditions did you test for in the calendar logic?

A: "I tested standard mid-month dates, the last day of the month where remaining days should equal exactly 1 (meaning the daily limit should equal the entire flexible pool), and leap-year boundaries. For instance, I set the fake system date to February 15th in 2028 (a leap year) and verified it counted 15 remaining days, and February 15th in 2029 (non-leap year) and verified it counted 14."

Q3: How does the system handle negative pools? What did the tests assert?

A: "If a user overspends or locks commitments exceeding their income, the flexible pool goes negative. To prevent the user interface from showing a negative daily budget, I asserted that the finance engine caps the daily spend limit at exactly zero, returning `Math.max(limit, 0)`. The test validated that even with a $-\$5,000$ pool, the daily limit returned was $\$0$."

Q4: How did you model mid-cycle budget changes in the unit tests?

A: "I simulated a user lifecycle within a single test. First, I ran the calculation on July 15th with $-\$5,000$ spent and $-\$5,000$ locked in goals, verifying a limit of $\$1,176$. Then, I simulated the user locking an additional $-\$10,000$ in a goal mid-month. The test passed when the engine dynamically recalculated the flexible pool and asserted the daily limit successfully dropped to $\$588$."

Q5: What value did these tests add to the codebase?

A: "They guaranteed that core calculations—which dictate the main metrics shown to the user on the dashboard—remain 100% accurate. It prevents regressions if another developer edits the finance helper or updates dependencies, and it documents the financial logic rules as executable code."

